ISLAMIC CALL UNIVERSITY COLLEGE

TITLE: CAMPUSMART-STUDENT MARKETPLACE SYSTEM ANALYSIS & DESIGN DOCUMENT

COURSE UNIT: MOBILE APPLICATION & DESIGN

REFERENCE PLATFORMS: Jiji Uganda and ZiBuy Uganda

YEAR: 3

SEMESTER: 1

SESSION: WEEKEND

Group Members

NAME	REG NO:	COURSE	SESSION
MUWONGE ABBASI	241-21301-7261	BCS	Weekend
WAZIRI ASLAM	241-22302-7521	BIT	Weekend
KAMARA YUSUF	241-22302-7631	BIT	Weekend

1. Problem Statement

Students often buy and sell textbooks, phones, laptops, hostel items, clothes and stationery. Some also offer services such as printing, laundry, and tutoring. These offers are commonly shared in class WhatsApp groups or on notice boards, where posts can be hard to find later. It can also be difficult to search by price or know whether a seller belongs to the student community. General marketplaces serve many types of users, so students may have trouble finding nearby items that suit their needs.

Campus Mart is a mobile app for students who want to buy or sell items within the campus community. A student creates an account and posts an item with its photo, price, and details. Other students can browse by category, search and filter listings, save favorites, message the seller, and send a purchase request. The seller can accept or reject the request, after which both students can agree on a suitable campus meeting point. The app will be built with Flutter and will connect to a REST API and a relational database.

Objectives

1. Give students one searchable place to list and find items on and around campus.
2. Make trading safer through registered accounts, visible seller names and campuses, and campus pickup points.
3. Practice the full chain taught in the course: analysis, ERD, REST API, Flutter app, and teamwork in Git.

Scope. In scope: accounts and login, listings, search and filters, favorites, purchase requests, messages, and basic admin tools.

Out of scope: online payment and delivery. Payment is cash or mobile money handed over at the meet-up, outside the app.

2. Team, Roles and Technology

Member	Role	Owns	Also expected to
Kamara Yusuf	Backend lead	API, database, run instructions	Review the app's API calls
Muwonge Abbasi	Flutter lead	Screens, navigation, forms	Review the backend's responses
Waziri Aslam	Analyst and integration lead	System analysis, ERD, API documentation, testing, README	Write code in both the app and the backend

All three group members are expected to contribute code and push regular commits to GitHub, at least once each week.

Layer	Choice	Why
Mobile app	Flutter (Dart), Provider for state, http package, flutter_secure_storage for the token, image_picker for photos	Taught in class; one codebase for Android and iOS
Backend	FastAPI (Python) with SQLAlchemy	Taught in class; automatic API docs at /docs

Layer	Choice	Why
Database	SQLite while developing (can move to PostgreSQL)	Allowed by the guidelines; no server to install
Authentication	bcrypt password hashes (passlib) and JWT tokens (python-jose)	Required by the guidelines
Documentation and tools	Swagger /docs, Postman, dbdiagram.io, Figma, GitHub	Required deliverables and teamwork

3. Users and Their Goals

Actor	Description	Main goals
Guest	Not logged in	Register, log in, see what the app offers
Student	Registered user who can be both buyer and seller	Post and manage items, search and filter, save favorites, chat, send and answer purchase requests, track orders
Admin	Group-appointed moderator	Manage categories, remove inappropriate listings

4. Requirements

4.1 Functional requirements

ID	The system shall...
FR1	let a guest register with full name, email, phone, campus and a password of at least 8 characters
FR2	reject registration when the email already exists (400)
FR3	let a user log in with email and password and return a JWT access token
FR4	store passwords only as bcrypt hashes, never as plain text
FR5	return 401 on protected endpoints when the token is missing or invalid
FR6	clear the token from the phone when the user logs out
FR7	let a user view and edit their profile (name, phone, campus)
FR8	let a user delete their account together with their own listings, favorites and messages
FR9	let a logged-in user post an item with title, description, category, price in UGX, condition, pickup location and one photo
FR10	validate listings: title 3 to 80 characters, description up to 500, price above 0, category must exist
FR11	show available items in a paged list (20 per page)
FR12	search items by keyword in the title and description
FR13	filter items by category, price range, condition and campus

ID	The system shall...
FR14	sort items by newest, price low to high and price high to low
FR15	show item details with the seller's name, campus and contact details
FR16	let a seller edit only their own items
FR17	let a seller delete only their own items
FR18	let a seller mark an item as available, reserved or sold
FR19	show a seller their own listings in My Listings
FR20	let a user add an item to favorites, remove it, and view all favorites
FR21	let a buyer send a purchase request for an available item, but not for their own item (400)
FR22	show a seller the requests received and let them accept or reject each one
FR23	show a buyer their orders with status and let them cancel while the request is pending
FR24	let the seller mark an accepted order as completed after the meet-up, which sets the item to sold
FR25	let a user send a message to a seller about an item and view the conversation
FR26	show an inbox with one entry per conversation
FR27	let an admin create, edit and delete categories
FR28	let an admin remove any inappropriate listing
FR29	ensure each user sees only their own orders, favorites and messages

4.2 non-functional requirements

ID	Requirement
NFR1	Usability: thumb-friendly bottom navigation, touch targets of at least 44 px, clear labels, and the same layout rules on every screen
NFR2	Performance: lists load in under 3 seconds on a slow connection, using paging (20 per page) and compressed photos
NFR3	Reliability: when the server is off or the network drops, the app shows a clear error with a Retry button and never crashes
NFR4	Security: HTTPS in production, bcrypt hashing, JWT on every protected request, and input validation on the server as well as in the app
NFR5	Privacy: a seller's phone number is visible only to logged-in users, and password hashes are never returned by the API
NFR6	Data integrity: foreign keys, unique email, and check constraints in the database, so the tables always match the ERD

ID	Requirement
NFR7	Maintainability: Flutter code organized into models/, services/, screens/, and widgets/, and backend code split into routers, models, and schemas
NFR8	Portability: The README lists the exact steps to run the backend and the app on a laptop

5. User Stories

1. As a guest, I want to register with my email and campus so that I can buy and sell.
2. As a student, I want to log in and stay logged in so that I do not retype my details every time.
3. As a seller, I want to post an item with a photo and a price so that other students can find it.
4. As a seller, I want to edit or delete my listing so that it always shows the right price and details.
5. As a seller, I want to mark an item as sold so that buyers stop asking for it.
6. As a buyer, I want to search and filter by category, price and condition so that I find items quickly.
7. As a buyer, I want to save items to favorites so that I can compare them later.
8. As a buyer, I want to message the seller about an item so that I can ask questions and agree on a meeting point.
9. As a buyer, I want to send a purchase request so that the seller knows I am serious.
10. As a seller, I want to accept or reject requests so that I choose who I meet.
11. As a buyer, I want to cancel a pending request so that I can change my mind.
12. As an admin, I want to manage categories and remove bad listings so that the marketplace stays clean.
13. As a student, I want to delete my account so that my data is removed.

Use case summary.

Guest: register, log in, see what the app offers

Student (buyer): search, filter, view item, favorite, message, send request, cancel request.

Student (seller): post, edit, delete, mark sold, accept or reject request, complete order.

Admin: manage categories, remove listings.

6. Entity Relationship Diagram (ERD)

The database is planned around six tables: users, categories, items, orders, favorites, and messages. Each table has a primary key, and foreign keys connect related records. The design aims to avoid storing the same information in several places. For example, a seller's campus is stored in the users table rather than repeated in every item listing. The favorites table links users and items, allowing one user to save many items and one item to be saved by many users.

6.1 Relationships and cardinality

Relationship	Cardinality	Foreign key
One user posts many items	1:N	items.seller_id references users.user_id
One category contains many items	1:N	items.category_id references categories.category_id
One user places many orders (as buyer)	1:N	orders.buyer_id references users.user_id

Relationship	Cardinality	Foreign key
One item can receive many orders (purchase requests)	1:N	orders.item_id references items.item_id
Users save many items and an item is saved by many users	M:N, resolved by favorites	favorites.user_id and favorites.item_id (composite primary key)
One user sends many messages	1:N	messages.sender_id references users.user_id
One user receives many messages	1:N	messages.receiver_id references users.user_id
One item can be the topic of many messages	1:N	messages.item_id references items.item_id

6.2 ERD diagram

The ERD shows the six planned tables and the main relationships between users, listings, categories, orders, favourites and messages.

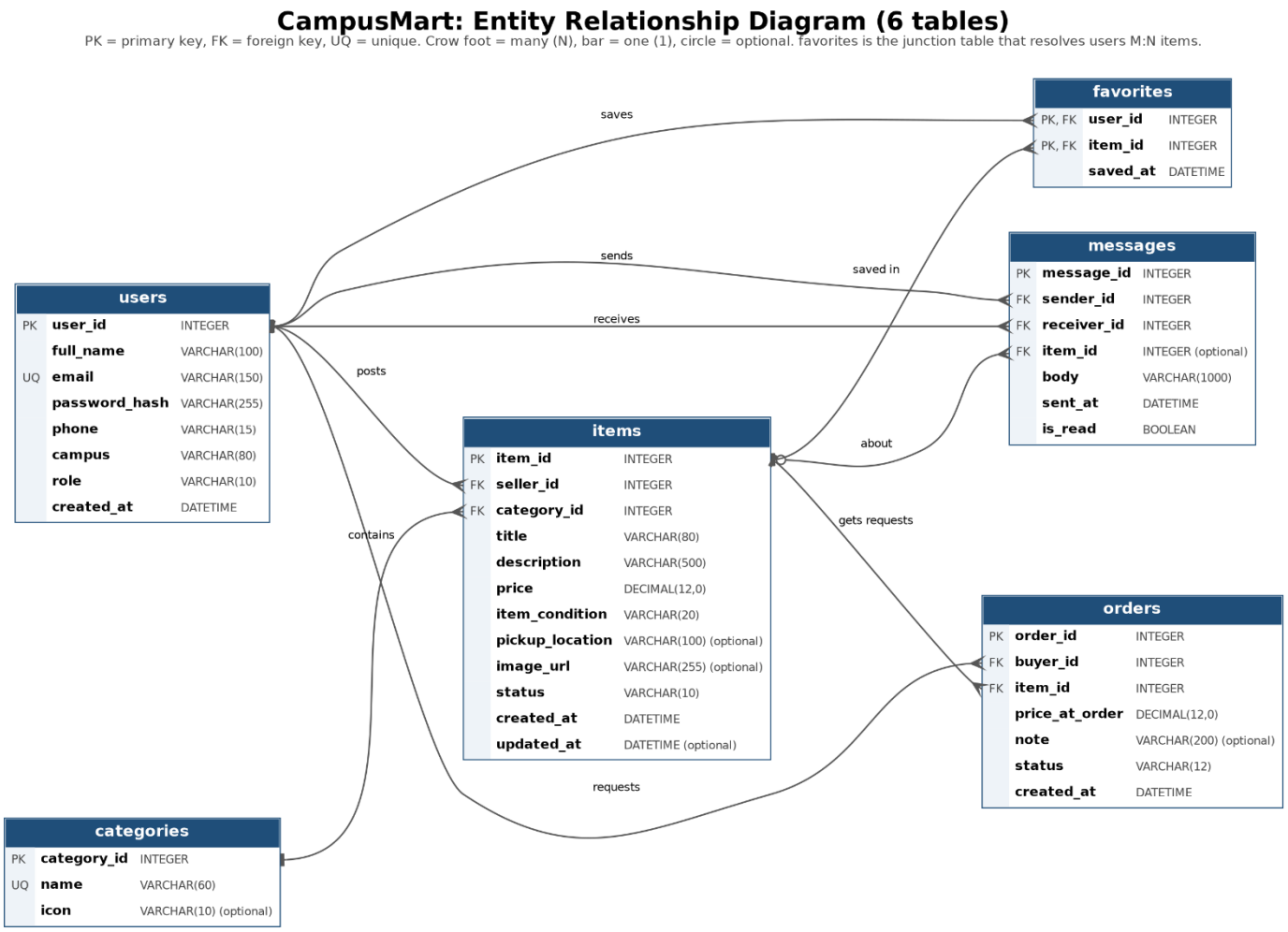


Figure A. Campus Mart entity relationship diagram.

```
Table users {
  user_id int [pk, increment]
  full_name varchar(100) [not null]
  email varchar(150) [not null, unique]
  password_hash varchar(255) [not null]
  phone varchar(15) [not null]
  campus varchar(80) [not null]
  role varchar(10) [not null, default: 'student']
  created_at datetime [not null]
}

Table categories {
  category_id int [pk, increment]
  name varchar(60) [not null, unique]
  icon varchar(10)
}

Table items {
  item_id int [pk, increment]
  seller_id int [not null, ref: > users.user_id]
  category_id int [not null, ref: > categories.category_id]
  title varchar(80) [not null]
  description varchar(500) [not null]
  price decimal(12,0) [not null]
  item_condition varchar(20) [not null]
  pickup_location varchar(100)
  image_url varchar(255)
  status varchar(10) [not null, default: 'available']
  created_at datetime [not null]
  updated_at datetime
}

Table orders {
  order_id int [pk, increment]
  buyer_id int [not null, ref: > users.user_id]
  item_id int [not null, ref: > items.item_id]
  price_at_order decimal(12,0) [not null]
  note varchar(200)
  status varchar(12) [not null, default: 'pending']
  created_at datetime [not null]
}

Table favorites {
  user_id int [not null, ref: > users.user_id]
  item_id int [not null, ref: > items.item_id]
  saved_at datetime [not null]
  indexes {
    (user_id, item_id) [pk]
  }
}

Table messages {
  message_id int [pk, increment]
  sender_id int [not null, ref: > users.user_id]
  receiver_id int [not null, ref: > users.user_id]
```

```
  item_id int [ref: > items.item_id]
  body varchar(1000) [not null]
  sent_at datetime [not null]
  is_read boolean [not null, default: false]
}
```

7. Data Dictionary

PK = primary key, FK = foreign key, UQ = unique, NN = not null.

users (one row per student or admin)

Field	Type	Size	Constraints	Meaning
user_id	INT		PK, auto increment	Unique id of the user
full_name	VARCHAR	100	NN	Name shown on listings
email	VARCHAR	150	NN, UQ	Login name
password_hash	VARCHAR	255	NN	bcrypt hash, never returned by the API
phone	VARCHAR	15	NN	Contact number, format 2567XXXXXXXX
campus	VARCHAR	80	NN	University or campus the student belongs to
role	VARCHAR	10	NN, default student	student or admin
created_at	DATETIME		NN	Registration time

categories (seeded: Books, Electronics, Phones, Clothes and Shoes, Hostel and Furniture, Stationery, Food and Snacks, Services, Other)

Field	Type	Size	Constraints	Meaning
category_id	INT		PK, auto-increment	Unique id
name	VARCHAR	60	NN, UQ	Category name
icon	VARCHAR	10		Emoji shown on the category chip

items (one row per listing; each listing is one item)

Field	Type	Size	Constraints	Meaning
item_id	INT		PK, auto-increment	Unique id
seller_id	INT		FK to users, NN	Who posted it
category_id	INT		FK to categories, NN	Category of the item
title	VARCHAR	80	NN, at least 3 characters	Short name

Field	Type	Size	Constraints	Meaning
description	VARCHAR	500	NN	Details of the item
price	DECIMAL	12,0	NN, above 0	Price in UGX
item_condition	VARCHAR	20	NN	new, like new, used good or used fair
pickup_location	VARCHAR	100		Meeting point on campus, e.g. Main Gate
image_url	VARCHAR	255		Path of the uploaded photo
status	VARCHAR	10	NN, default available	available, reserved or sold
created_at	DATETIME		NN	Time posted
updated_at	DATETIME			Time last edited

orders (a purchase request from a buyer for an item)

Field	Type	Size	Constraints	Meaning
order_id	INT		PK, auto-increment	Unique id
buyer_id	INT		FK to users, NN	Who wants to buy
item_id	INT		FK to items, NN	The requested item (the seller is found through the item)
price_at_order	DECIMAL	12,0	NN	Price when the request was sent, kept even if the seller later edits the price
note	VARCHAR	200		Message to the seller, e.g. preferred meeting time
status	VARCHAR	12	NN, default pending	pending, accepted, rejected, cancelled or completed
created_at	DATETIME		NN	Time of request

favorites (junction table between users and items)

Field	Type	Size	Constraints	Meaning
user_id	INT		PK part 1, FK to users	Who saved the item
item_id	INT		PK part 2, FK to items	The saved item
saved_at	DATETIME		NN	Time saved

messages

Field	Type	Size	Constraints	Meaning
message_id	INT		PK, auto increment	Unique id
sender_id	INT		FK to users, NN	Who sent it
receiver_id	INT		FK to users, NN	Who receives it
item_id	INT		FK to items	The item being discussed
body	VARCHAR	1000	NN	Message text
sent_at	DATETIME		NN	Time sent
is_read	BOOLEAN		NN, default false	Whether the receiver has opened it

Business rules. A buyer cannot send a request for their own item. When a request is accepted the item becomes reserved; when it is completed the item becomes sold; when it is rejected or cancelled the item becomes available again. Deleting a user deletes their items, favourites and messages.

8. API Endpoint Table

The API returns JSON over HTTP and is documented automatically by FastAPI at /docs (Swagger). Protected endpoints need the header Authorization: Bearer <token> and return 401 when the token is missing or invalid. Status codes used: 200 OK, 201 Created, 400 validation error, 401 unauthorised, 403 not the owner, 404 not found.

Method	Path	Access	Purpose	Request body	Response
POST	/auth/register	Public	Create account	full_name, email, phone, campus, password	201 user; 400 if email exists or data invalid
POST	/auth/login	Public	Log in	email, password	200 access_token and user; 401 wrong details
GET	/auth/me	Token	Current user	none	200 user; 401
PUT	/users/me	Token	Edit own profile	full_name, phone, campus	200 user; 400
DELETE	/users/me	Token	Delete own account	none	200 message
GET	/categories	Public	List categories	none	200 list
POST	/categories	Admin	Create category	name, icon	201 category; 400 duplicate
PUT	/categories/{id}	Admin	Edit category	name, icon	200 category; 404
DELETE	/categories/{id}	Admin	Delete category	none	200; 400 if items use it; 404

Method	Path	Access	Purpose	Request body	Response
GET	/items	Public	List available items	query: q, category_id, min_price, max_price, condition, campus, sort, page	200 list
GET	/items/{id}	Public	Item details with seller	none	200 item; 404
POST	/items	Token	Post an item	category_id, title, description, price, item_condition, pickup_location	201 item; 400
PUT	/items/{id}	Owner	Edit own item	same as POST	200 item; 403; 404
PATCH	/items/{id}/status	Owner	Mark available, reserved or sold	status	200 item; 400
DELETE	/items/{id}	Owner or admin	Delete item	none	200; 403; 404
POST	/items/{id}/image	Owner	Upload photo	multipart file	200 image_url; 400 wrong file type
GET	/users/me/items	Token	My listings	none	200 list
GET	/favorites	Token	My favourites	none	200 list of items
POST	/favorites	Token	Save an item	item_id	201; 400 already saved; 404
DELETE	/favorites/{item_id}	Token	Remove from favourites	none	200; 404
POST	/orders	Token	Send purchase request	item_id, note	201 order; 400 own item or not available
GET	/orders/mine	Token	Orders I placed	none	200 list
GET	/orders/received	Token	Requests for my items	none	200 list
PATCH	/orders/{id}/status	Buyer or seller	Buyer cancels; seller accepts, rejects or completes	status	200 order; 400 invalid change; 403; 404

Method	Path	Access	Purpose	Request body	Response
POST	/messages	Token	Send a message	receiver_id, item_id, body	201 message; 400
GET	/messages/conversations	Token	Inbox	none	200 list
GET	/messages/thread	Token	Conversation about an item	query: with_user, item_id	200 list

9. Screen Sketches

The proposed app has 12 screens. The screen sketches should be saved in the docs/ folder. A bottom navigation bar will provide access to Home, Favorites, Sell, Orders and Profile. Users can open messages from the home screen or an item's details page.

Screen	What the user sees	Key actions and states
1. Welcome	Logo, short tagline, Log in and Register buttons	Skips to Home if a valid token is stored
2. Register	Full name, email, phone, campus, password and confirm password	Form validation, loading spinner, error message if the email exists
3. Login	Email and password with show/hide icon	Validation, error for wrong details, loading state
4. Home	Search bar, category chips, filter button, grid of item cards (photo, title, price, campus)	Pull to refresh, paging, empty state, error state with Retry when the server is off
5. Filter and sort sheet	Price range, condition, campus, sort options	Apply and Reset
6. Item details	Photo, title, price, condition, description, pickup point, seller name and campus, favourite heart	Message seller, Send purchase request, Edit and Delete (owner only)
7. Post or edit item	Photo picker, title, description, category dropdown, price, condition, pickup location	Field validation, upload progress, Save
8. My listings	The seller's items with status labels	Edit, delete, mark sold
9. Favourites	Saved items	Remove from favourites, open item
10. Orders	Two tabs: My Orders and Requests Received	Cancel (buyer), Accept, Reject, Complete (seller)
11. Messages	Inbox list and a chat thread	Send message, unread dot
12. Profile	Name, email, phone, campus	Edit profile, Log out (clears the token), Delete account

Bonus if time allows: an admin screen for categories and removing listings, offline saving of the last Home page, and a Contact on WhatsApp button on Item details.

10. Project Plan and Milestones

Dates	Milestone	Waziri Aslam (backend)	Muwonge Abbasi (Flutter)	Kamarayusuf (analysis and integration)
By 24 Oct	Design document submitted as a PDF in docs/	Review ERD and endpoint table	Draw screen sketches	Finish document, ERD image, push to GitHub
25 Oct to 8 Nov	Backend running	Database tables, register and login with JWT, items, favorites, orders, messages	Project structure, screens 1 to 7 with sample layouts, navigation	Swagger documentation, Postman collection, test each endpoint, README run steps
8 to 15 Nov	App connected	Fix API issues, image upload, validation	Services and models, real data on every screen, loading and error states	Integration testing between app and API, bug list
15 to 20 Nov	Polish and code freeze	Final checks, seed data	Screens 8 to 12, form validation, logout	README with screenshots, final Git cleanup
20 Nov	Repository link submitted	All three members have regular commits		
23 Nov	Demo, about 10 minutes	Explain API and database	Explain app code	Pitch, ERD next to the real tables

Repository structure

```
campusmart/
  backend/   FastAPI project and its README with run steps
  app/       Flutter project (lib/models, lib/services, lib/screens, lib/widgets)
  docs/      design document PDF, ERD image, API documentation
  README.md  group names, project idea, screenshots
```

Demo checklist. Start the server and app before the turn; show register, login, create, view, edit and delete; show the ERD beside the real database tables; keep a short screen recording as a backup; make sure each member can answer a question about the code.

11. Risks

Risk	What we will do
Too many features for three weeks	Finish the core first (login, items, orders); treat chat, favourites polish and admin as lower priority
App and backend do not match	Agree on the endpoint table in section 8 before coding; change it in the document first
One member does most of the commits	Everyone commits at least weekly with clear messages; small tasks per person
Photo upload fails on slow networks	Compress photos in the app and show upload progress and a clear error
Demo network fails	Run the server on the laptop, and keep a screen recording as backup

Appendix A: Screen Sketches

These sketches show the proposed layout of the main CampusMart screens. They are early design references and may change during development.

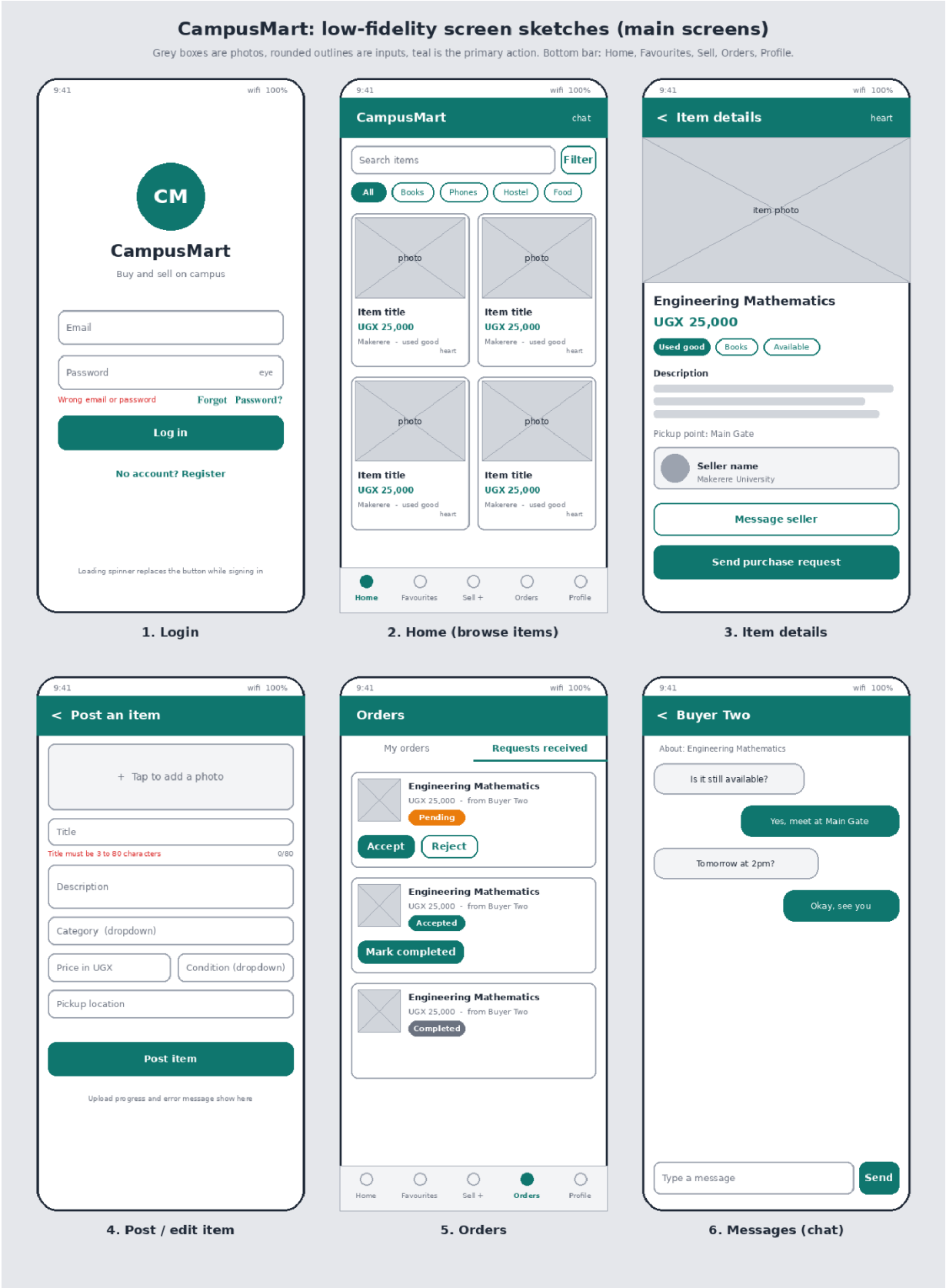


Figure B. Main screen sketches for CampusMart